JAVASCRIPT

Einführung in die Programmierung mit JavaScript

© Foliensatz (Stephan Diehl, Jascha Knack, Michael Fritz)

ÜBERBLICK

- **Vorstellung**
- **Allgemeines**
  - Was ist JavaScript?
  - Wie und wo wird programmiert? - Die Entwicklungsumgebung
- **Programmierung**
  - Aufbau eines Programms
  - Variablen, Datentypen & Ausdrücke — *~ 1. Übung: Addition zweier Zahlen ~*
  - Bedingungen: Die IF-Anweisung — *~ 2. Übung: "Taschenrechner" ~*
  - Schleifen: FOR, WHILE & DO — *~ 3. Übung: Summenberechnung ~*
  - Methoden — *~ 4. Übung: Methoden auslagern ~*
  - Objek<te — *~ 5. Übung: Objekt Vector3 implementieren*

WAS IST JAVASCRIPT

JavaScript ist eine Skriptsprache

Früher interpretiert, heute JIT-kompiliert

Entwickelt für dynamische Elemente in Websites Interaktion mit dem Benutzer, Verändern von Inhalten)

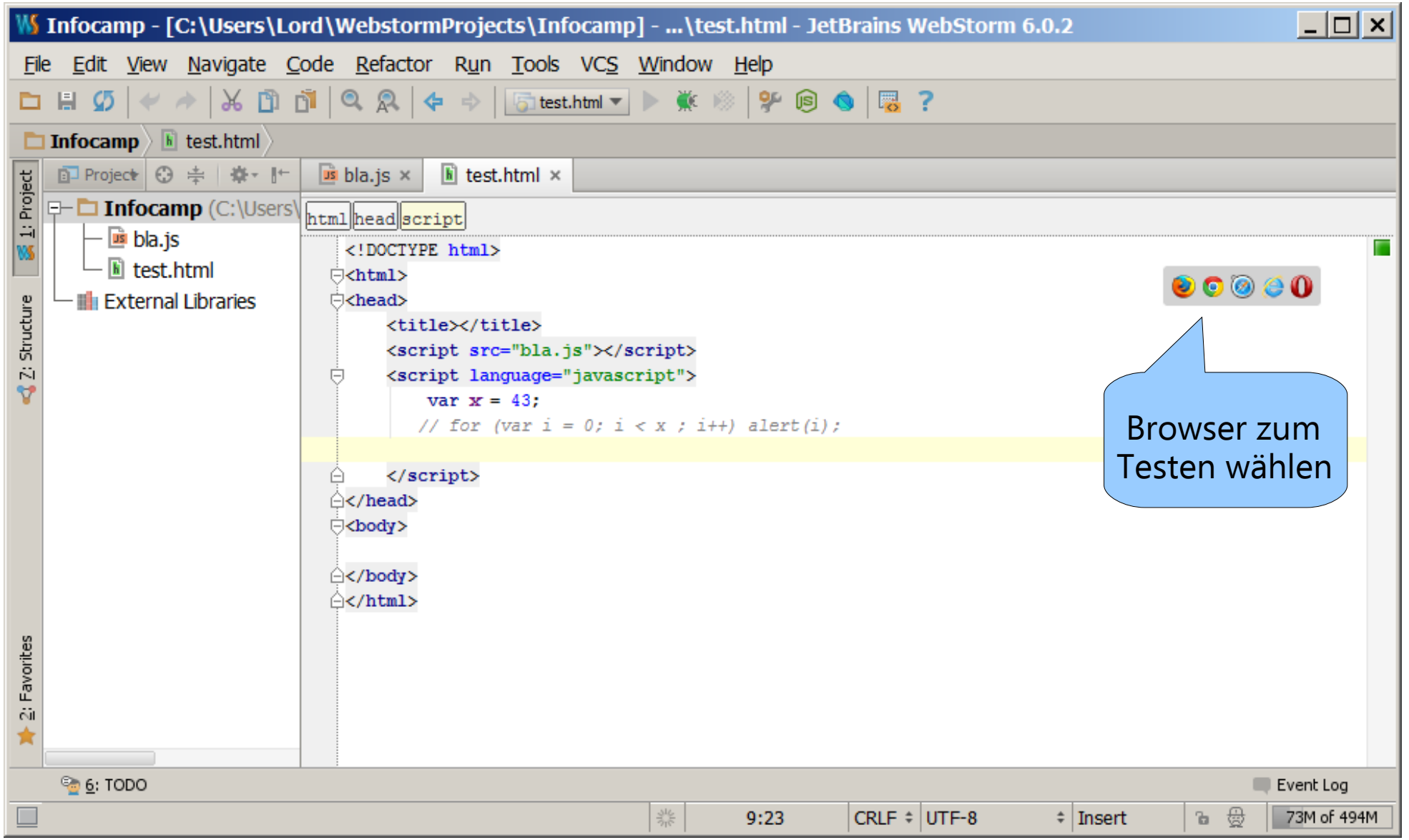
Verwendung heutzutage nicht nur noch in Browsern, sondern auch z.B. in Microcontrollern oder auf Servern

JavaScript Ist NICHT Java. JavaScript hieß eigentlich LiveScript und war Bestandteil des Netscape Browsers. Um auf den “Java-Zug” aufzuspringen, wurde die Sprache dann umbenannt.

Entwicklungsumgebung



Entwicklungsumgebung



Grundlegendes

Aufbau des Codes der HTML-Seite:

```
<html>
<head>
  <title>Infocamp 2018</title>
  <script>
     // JavaScript Anweisungen
  </script>
</head>
<body>
    // HTML Anweisungen
</body>
</html>
```

Jede JavaScript-Anweisung mit einem **Semikolon** abschließen!

Beispiel:

```
var x = 3;
Methodenaufruf();
```

Grundlegendes

Importieren eines Skripts in HTML:

```
<html>
<head>
  <title>Infocamp 2016</title>
  <script src="test.js"></script>
  <script src="test2.js"></script>
  <script>
      // JavaScript Anweisungen
  </script>
</head>
<body>
    // HTML Anweisungen
</body>
</html>
```

Mehrere Script-Tags möglich!

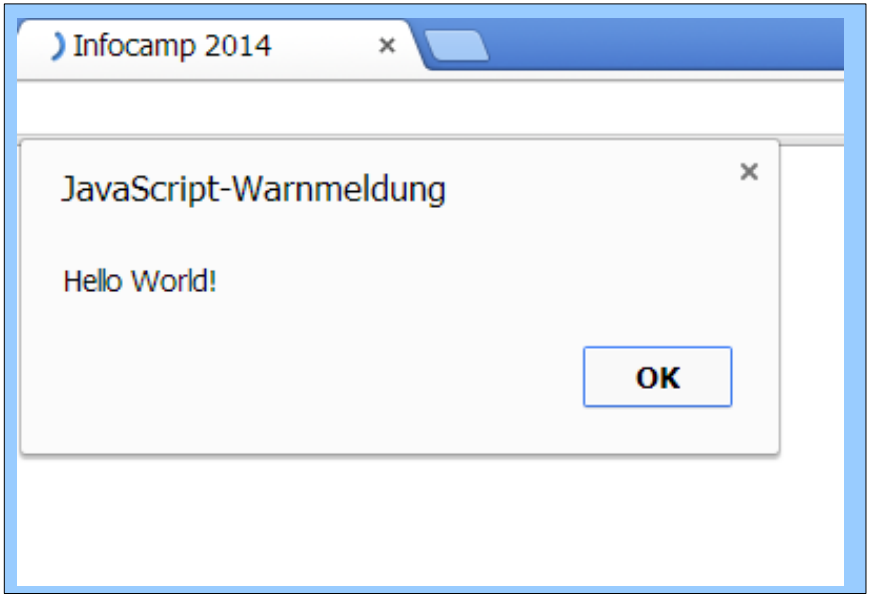
Jede JavaScript-Anweisung mit einem **Semikolon** abschließen!

Beispiel:

```
var x = 3;
Methodenaufruf();
```

Hello World!

```
<html>
<head>
  <title>Infocamp 2018</title>
  <script>
     alert("Hello World!");
  </script>
</head>
<body>
    // HTML Anweisungen
</body>
</html>
```



Ziel der Einführung

Entwickeln eines einfachen Taschenrechners mit den Grundrechenarten und ein paar weiteren Funktionen.

Zuerst: Simpler Addierer

- ✗ Eingabe von 2 Zahlen
- ✗ Berechnen der Summe
- ✗ Ausgabe der Summe

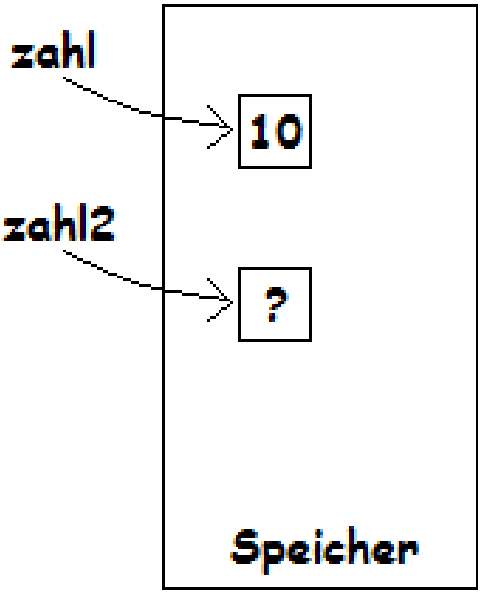
Was wir benötigen:

- Werte speichern
- Rechnen mit diesen Werten

Variablen: Einfache Datentypen

Initialisierung

```
var zahl = 10;
var zahl2;
var test = true;
var x = "Text usw.";
```



Verwendung:

```
alert(zahl);
zahl = 3;
zahl2 = zahl;
```

Variablen: Array-Datentyp

Initialisierung:

var *numbers =* ***new Array(5)****;*

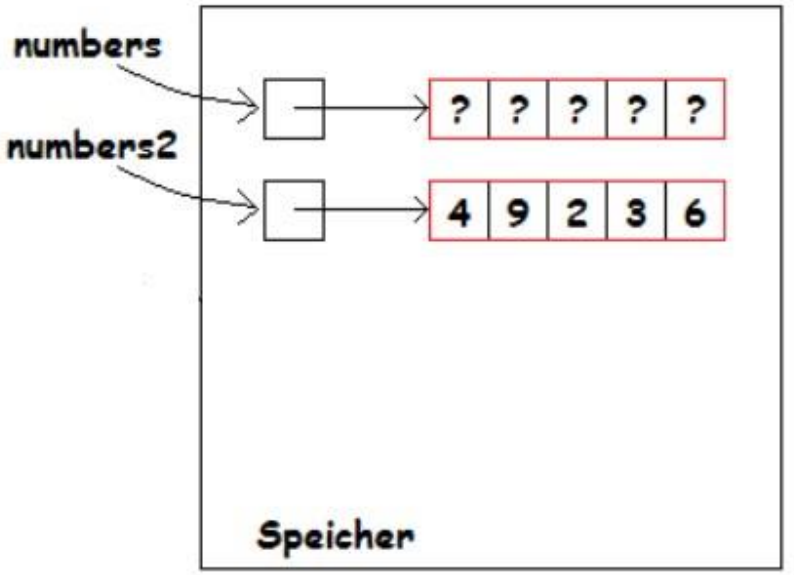
var *numbers2 = [4,9,2,3,6];*

Verwendung:

alert(numbers2[0]);

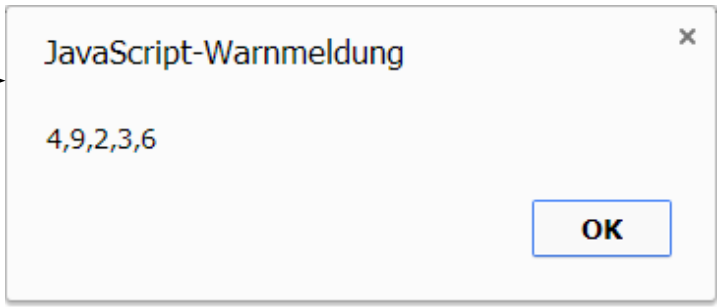
numbers[4] = 29;

numbers2[0] = numbers2[2];



Ausgabe eines Arrays: alert(numbers2);

numbers.length liefert Länge des Arrays!



Variablen: String

Hinweis: Datentyp Object wird erst später vorgestellt.

Initialisierung:

```
var x = "text123+#2";
var y = "";
```

Sonderzeichen:

```
x = "C:\\Ordner - \"Hallo\" ";
```

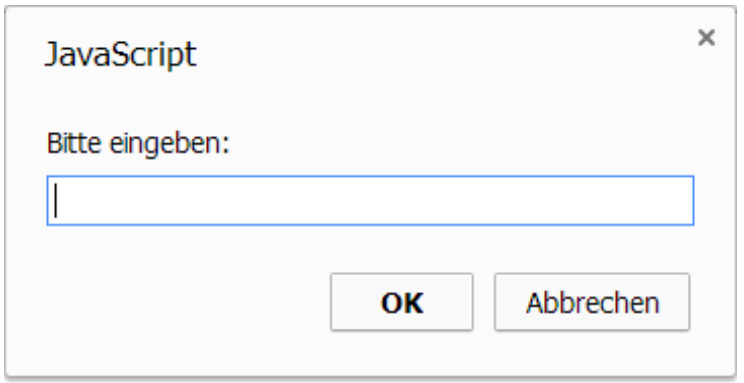
\n - Zeilenumbruch

- *x.length* gibt die Länge einer Zeichenkette an
- ist Variable keine Zeichenkette dann "undefined"
- einzelne Zeichen des Strings wie beim Array per Index aufrufen:

```
alert(x[3]);
// Gibt ein t aus.
```

Benutzereingabe

- **Prompt-Methode**
  var eingabe = **prompt**("Bitte eingeben:");
  → Speichert die Benutzereingabe als Zeichenkette



- **Einlesen aus Textfeld der HTML-Seite**
  (später – *HTML & JavaScript*)

Ausdrücke

Seien gegeben:

```
var x = 1;
var y = 1.3;
var t = "Hallo";
var b = true;
```

x+1	*x+x*	*x+y*
*(2*x)+(y/2)*	*t+y*	*x % 2*
b || false	*b&&true*	*b!=true*

Operator + bei String-Datentyp: Konkatenation:

„Hintereinanderreihen“ von Werten, in diesem Fall Zeichenketten.

z.B.: *“Wurst“ + “brot“ → “Wurstbrot“*
“Hallo“ + 123 → “Hallo123“

Aber: *“Test“ + 4 + 2 → “Test42“*
4 + 2 + “Test“ → “6Test“

Math Objekt

Das Math-Objekt beinhaltet wichtige Funktionen und Konstanten aus der Mathematik.

Minimum & Maximum:

- Math.**min**(<wert1>,<wert2>); → Minimum der beiden Werte
- Math.**max**(<wert1>,<wert2>); → Maximum der beiden Werte

Runden:

- Math.**floor**(<wert>); → abrunden
- Math.**ceil**(<wert>); → aufrunden
- Math.**round**(<wert>); → ab x.5 aufrunden, sonst abrunden

Potenzen:

- Math.**sqrt**(<wert>); → Quadratwurzel (√<wert>)
- Math.**pow**(<basis>, <exponent>); → Potenzieren (<basis>$^{<exponent>}$)

Zufall:

- Math.**random**(); → Zufallszahl zwischen 0 und 1
  => var x = Math.**random**()*<wert>; → Zufallszahl zwischen 0 und <wert>

Konstanten:

- Math.**PI** → 3.141592653589793

Parse - Funktionen

Funktionen zum Konvertieren von **Zeichenketten** in entsprechende Werte anderer Datentypen

Zieltyp	Aufruf
Ganze Zahl (Integer bzw. Int)	*parseInt(<String>);*
Kommazahl (Float)	***parseFloat(<String>);***

```
var str = "123";
var str2 = 2;
alert(parseInt(str) + str2);
```

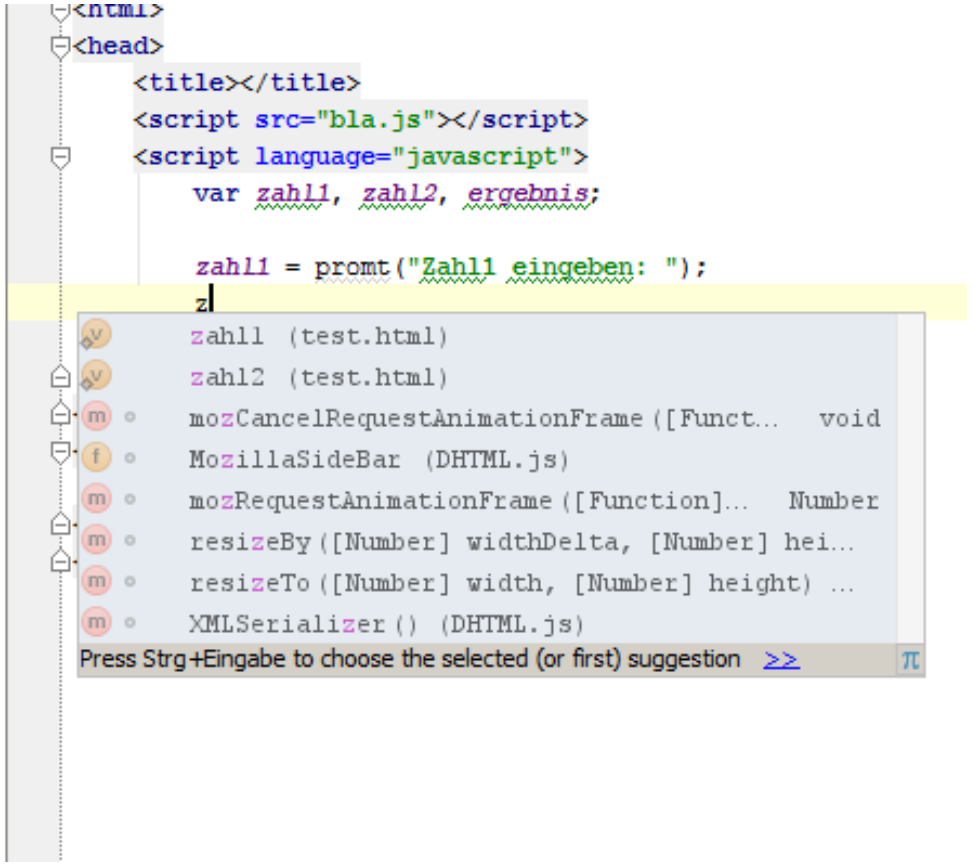
Ausgabe: 125

```
var str = "123";
var str2 = 2;
alert(str + str2);
```

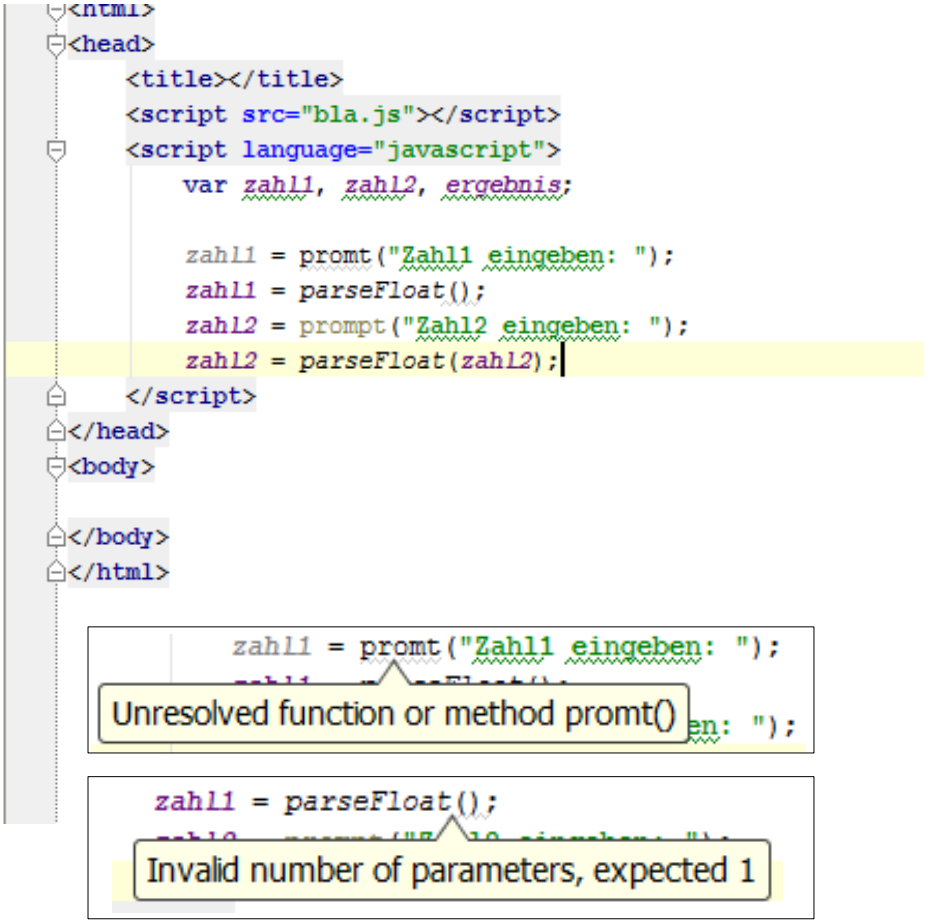
Ausgabe: 1232

Probleme?

Code Completion



Fehleranzeige



Anwendung: Addierer

```
<html>
<head>
  <title>Infocamp 2014</title>
  <script language="JavaScript">
        var zahl1, zahl2, ergebnis;

        // Einlesen des ersten Wertes
        zahl1 = prompt("Bitte Zahl 1 eingeben:");
        zahl1 = parseFloat(zahl1);

        // Einlesen des zweiten Wertes
        zahl2 = parseFloat(prompt("Bitte Zahl 2 eingeben:"));

        // Berechnen
        ergebnis = zahl1 + zahl2;

        // Ausgabe
        alert("Das Ergebnis ist : "+zahl1+" + "+zahl2+" = "+ergebnis);
    </script>
</head>
<body>
     <!-- HTML Anweisungen -->
</body>
</html>
```

Anlegen der Variablen

Einlesen einer Zeichenkette

Umwandeln von ***Zeichenkette*** in ***Zahl***

Berechnung durch Ausdruck

Ausgeben eines Ausdrucks

1. Übung – Variablen

Mögliche Projekte:

- ✓ Simpler Addierer: Eingabe zweier Zahlen, Ausgabe Ergebnis
- Eingabe Wort, ersten und letzten Buchstaben ausgeben
- Eingabe Zahl, mit zuf. Zahl zwischen 1 und 50 multiplizieren

Einfache Variablen:

var *x = 2;* ***var*** *y = 3.3;* ***var*** *a = x*y;* ***var*** *b = true;*

Array Initialisierung:

var *numbers =* ***new Array(****5);*
var *numbers2 = [4,9,2,3,6];*

Array Verwendung:

var *x = numbers.length;*
numbers[4] = 29;
(Indizes von 0 bis length-1!)

Umwandeln von Zeichenkette in Zahl:

var *z = parseFloat("123.5");*

Ausgabe: alert*(<wert>);*
Eingabe: **var** *e = prompt("Bitte Wert eingeben:");*

Nächster Schritt zum Rechner

Nun wollen wir alle 4 Grundrechenarten im Rechner unterbringen.

Was muss das Programm leisten? :

- Eingabe von 2 Zahlen
- Eingabe der gewünschten Rechenart
- Berechnung unter Bedingung der Eingabe
- Ausgabe des Ergebnisses

Was wir benötigen:

- Möglichkeit dem Programm Bedingungen mitzuteilen

Boolesche Ausdrücke

Ausdrücke, deren Ergebnis ein boolescher Wert ist.

Gleichheit:	Ungleichheit:
x === y Es wird *true* geliefert, wenn x gleich y ist; sonst *false*.	*x !== y* oder *!(x === y)* In beiden Fällen wird *true* geliefert, wenn x ungleich y ist; sonst *false*.

ACHTUNG: Zuweisung "="
Vergleich "===“ oder “==“

Besser ‘===’ und nicht ‘==’ verwenden!

==	gleicher Wert
===	gleicher Wert und gleicher Datentyp

Die Gleichheitsrelation in der Mathematik ist transitiv.
Wenn **a = b** und **b = c** dann ist auch **a = c**

Durch implizite Typkonvertierung der dynamisch typisierten Sprache JavaScript ist der == Operator jedoch weder kommutativ noch transitiv:

```
'' == '0'           // false
0 == ''             // true
0 == '0'            // true
```

Boolesche Ausdrücke

Ausdrücke, deren Ergebnis ein boolescher Wert ist.

Logisches „und“:
Nur mit 2 booleschen Werten bzw. Ausdrücken.
Seien x und y eben solche,
dann liefert *x && y*:

x	y	Ergebnis
false	*false*	*false*
false	*true*	*false*
true	*false*	*false*
true	*true*	*true*

Also:
x&&y ist *true*, wenn x **und** y *true* sind.

Logisches (einschließendes) „oder“:
Nur mit 2 booleschen Werten bzw. Ausdrücken.
Seien x und y eben solche,
dann liefert *x || y*:

x	y	Ergebnis
false	*false*	*false*
false	*true*	*true*
true	*false*	*true*
true	*true*	*true*

Also:
x||y ist *true*, wenn x **oder** y *true* ist.

Wie würde man nun ein ausschließendes "oder" (entweder, oder) realisieren?

(x||y) && !(x&&y) => x ^ y

Boolesche Ausdrücke

Zusammenfassung und Erweiterung:

Operatoren für boolesche Ausdrücke

Vergleich von Zahlenwerten:

x === y	*- Gleichheit*
x !== y	*- Ungleichheit*
x > y	*- größer als*
x >= y	*- größer gleich*
x < y	*- kleiner als*
x <= y	*- kleiner gleich*

Vergleich von booleschen Werten:

x === y	*- Gleichheit*
x !== y	*- Ungleichheit*
x && y	*- und*
x || y	*- oder*
x ^ y	*- entweder oder*
!x	*- Negation von x*

Prioritäten in Ausdrücken allgemein:

Klammern werden zuerst bearbeitet (wie in der Mathematik auch)
"Punkt- vor Strichrechnung"
Negationen werden vor anderen Berechnungen ausgeführt

() → ! → *,/,% → +,- → <,<=,>,>= → ===,!== → ^ → && → ||

Kontrollstrukturen: IF - Abfrage

if (<boolescher Ausdruck>) {
<Anweisung>;
}

oder

if (<boolescher Ausdruck>) {
<Anweisung>;
}
else {
<Anweisung>;
}

```
var x = 1;
var y = 2;
var z = false;
if (x ===y){
    // mach was
}

if (z){
    // mach was
}
else{
    // mach was anderes
}

if (true){
    // mach was
}

if (z || (x !==y)){
    // mach was
}
```

Kontrollstrukturen: IF - Abfrage

```
if (<boolescher Ausdruck>) {
        <Anweisung>;
}
else if (<boolescher Ausdruck>) {
        <Anweisung>;
}
else if (<boolescher Ausdruck>) {
        <Anweisung>;
}
else {
        <Anweisung>;
}
```

```
var a = 2;
var b = 1;
var c = 3;
if (a + b > c){
    alert("Summe ist größer.");
}
else if (a + b < c){
    alert("Summe ist kleiner.");
}
else if (a + b ==c){
    alert("Summe ist gleich.");
}
```

Geltungsbereiche von Variablen

Deklaration mit dem Schluesselwort <var> → function / global scope

```
var x = 1;
var y = 2;

if (x == y) {
    var a = 42;
    if (a != 3) {
        var b = 20;
        a += x;      // kurz für a=a+x;
        b += a;
    }
    alert(b);
}
```

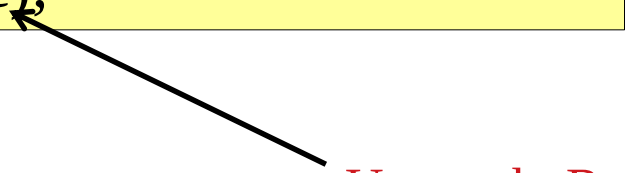
Das klappt!

Im Gegensatz zu Java!

Geltungsbereiche von Variablen

Deklaration mit dem Schluesselwort <let> → block scope

```
var a = 5;
var b = 5;
if(a === b) {
      let c = 7;
}
alert(c);
```



Uncaught ReferenceError: c is not defined

Wie in Java!

Geltungsbereiche von Variablen

Klappt das immer? Auch wenn a === 3?

```
var x = 1;
var y = 2;

if (x == y) {
    var a = 42;
    if (a != 3) {
        var b = 20;
        a += x;      // kurz für a=a+x;
        b += a;
    }
    alert(b);
}
```

Wenn a === 3, dann
b === undefined

Geltungsbereiche von Variablen

```
var x;
var y;
var a;
var b;
```

Das macht der Compiler intern und automatisch. Im Quellcode nicht sichtbar!

```
var x = 1;
var y = 2;

if (x == y) {
    var a = 42;
    if (a != 3) {
        var b = 20;
        a += x;      // kurz für a=a+x;
        b += a;
    }
    alert(b);
}
```

Wenn a === 3, dann b === undefined

Anwendung: Grundrechenarten

```
<html>
<head>
    <title>Infocamp 2014</title>
    <script language="JavaScript">
        var zahl1, zahl2, operator;
        var ergebnis = 0;

        // Einlesen beider Zahlen
        zahl1 = parseFloat(prompt("Zahl 1 eingeben:"));
        zahl2 = parseFloat(prompt("Zahl 2 eingeben:"));

        // Einlesen der Rechenart
        operator = prompt("Operator eingeben (+ - * /):");
        operator = operator[0];

        // Abfragen und entsprechend Berechnen
        if (operator == "+") { ergebnis = zahl1 + zahl2; }
        else if (operator == "-") { ergebnis = zahl1 - zahl2; }
        else if (operator == "*") { ergebnis = zahl1 * zahl2; }
        else if (operator == "/") { ergebnis = zahl1 / zahl2; }
        else { alert("Ungültiger Operator eingegeben."); }

        // Ausgabe
        alert(zahl1 + " " + operator + " " + zahl2 + " = " + ergebnis);
    </script>
</head>
<body>
    <!-- HTML Anweisungen -->
</body>
</html>
```

Abfragen des Operators

Nur erstes Zeichen der Eingabe berücksichtigen

else – Zweige für alle Möglichkeiten

Alle anderen Eingaben auch berücksichtigen!

2. Übung – IF, ELSE

Mögliche Projekte:

- ✓ kleiner Taschenrechner
- Maximum / Minimum zweier Zahlen (ohne Math-Objekt zu benutzen)
- Kleines Quiz (3 Fragen, Multiple Choice, richtige Antwort = 1 Punkt)

```
if (<boolescher Ausdruck>) {
        <Anweisung>;
}
else if (<boolescher Ausdruck>) {
        <Anweisung>;
}
else {
        <Anweisung>;
}
```

```
x === y  - Gleichheit
x !== y  - Ungleichheit
x > y    - größer als
x >= y   - größer gleich
x < y    - kleiner als
x <= y   - kleiner gleich
```

```
+, -, *, /  - bekannt ;-)
%           - Modulo
x && y      - log. und
x || y      - log. oder
x ^ y       - entweder oder
!x          - Negation von x
```

Weitere Funktionen des Rechners

Als nächstes wollen wir eine Funktion zum Berechnen der Summe der ersten x Zahlen einfügen.

Was muss das Programm leisten? :

- Eingabe einer Zahl x
- Berechnung der Summe bis Zahl x
- Ausgabe der Summe

Was wir benötigen:

- Möglichkeit die Addition beliebig oft oder in diesem Fall x-Mal hintereinander auszuführen

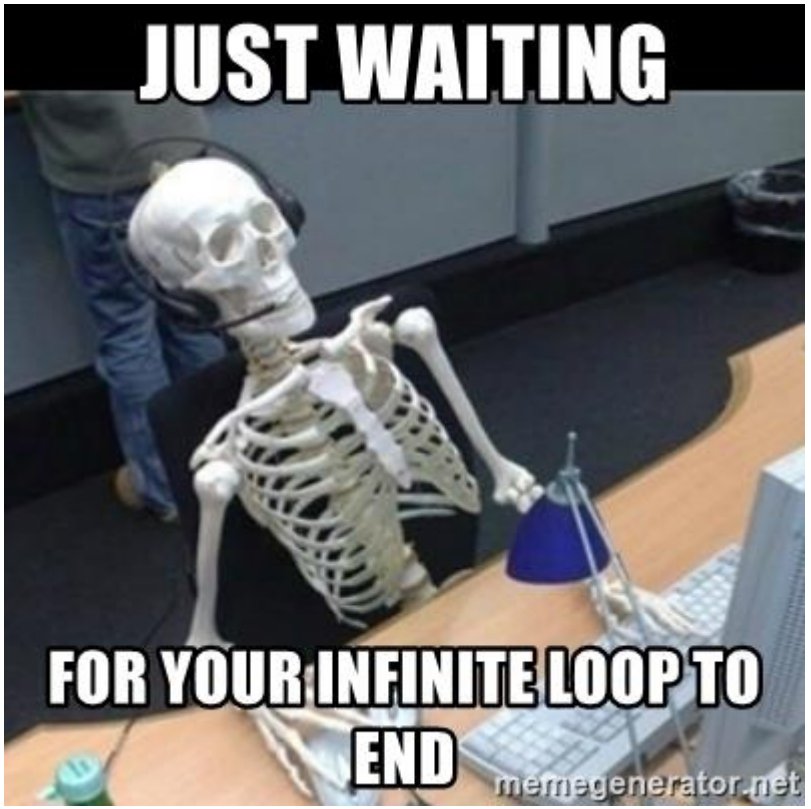
Schleifen

Schleifen dienen dazu, bestimmte Code-Abschnitte beliebig oft hintereinander auszuführen.

Beliebig oft heißt, die Programmstelle wird so oft wiederholt, bis eine vorher festgelegte Bedingung verletzt wird.

ACHTUNG: Endlosschleifen
Kann bzw. wird die oben erwähnte Bedingung niemals eintreffen,
so spricht man von einer Endlosschleife.

Dann hilft nur noch der Abbruch des Programms.



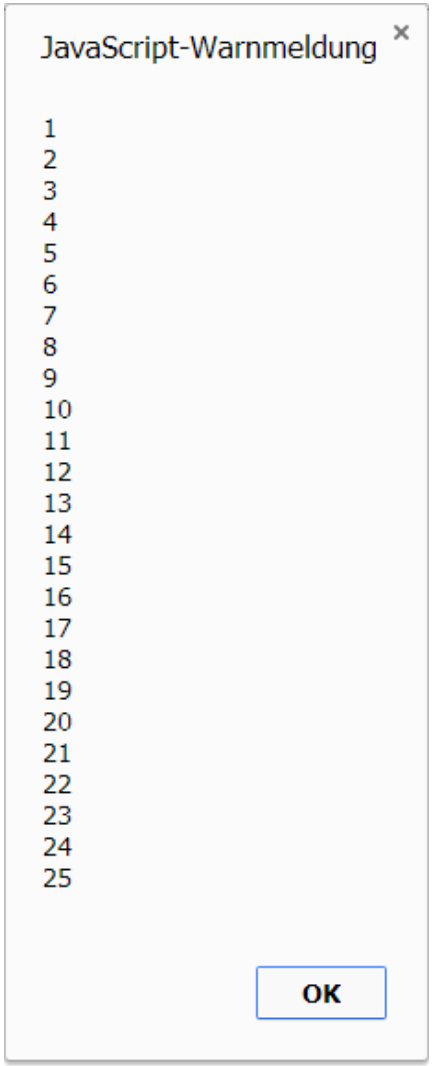
Die WHILE – Schleife

Aufbau:

```
while (<boolescher Ausdruck>) {
        <Anweisungen>;
}
```

Beispiel:

```
var a = 0, ausgabe = "";
while (a < 25) {
    a += 1;
    ausgabe += a+"\n";
}
alert(ausgabe);
```



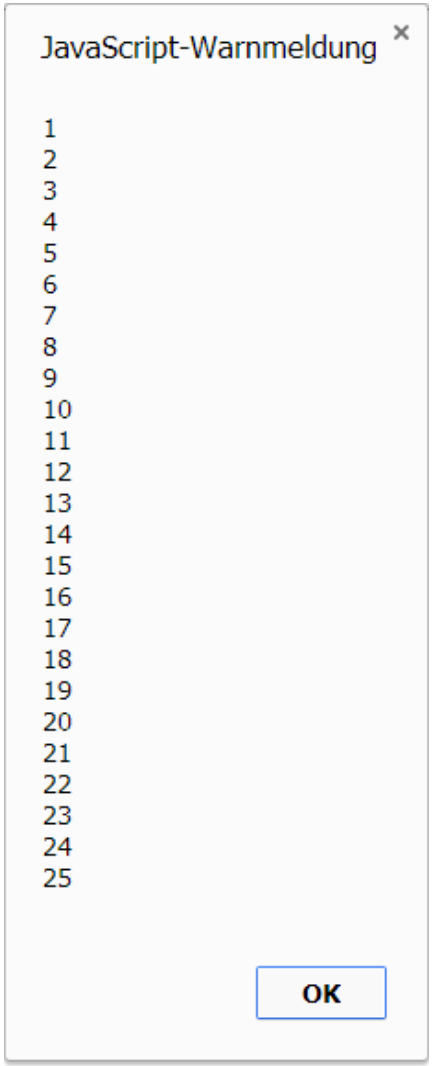
Die DO – Schleife

Aufbau:

```
do
{
      <Anweisungen>;
}while (<boolescher Ausdruck>);
```

Beispiel:

```
var a = 0, ausgabe = "";
do {
    a += 1;
    ausgabe += a+"\n";
}while (a < 25);
alert(ausgabe);
```



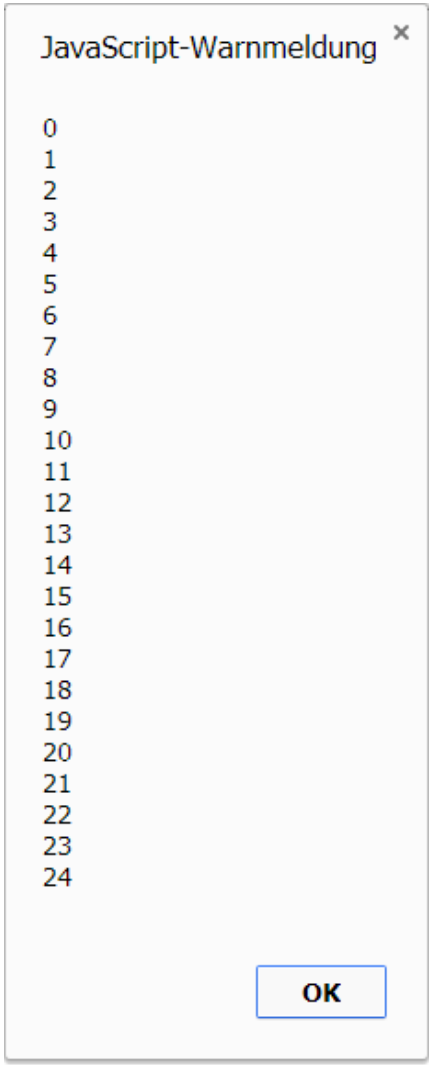
Die FOR – Schleife

Aufbau:

```
for (<Initialisierung>;<Bedingung>;<Fortsetzung>){
      <Anweisungen>;
}
```

Beispiel:

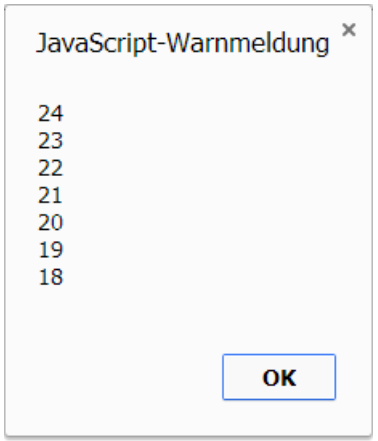
```
var ausgabe = "";
for (var a = 0; a < 25; a++) {
    ausgabe += a+"\n";
}
alert(ausgabe);
```



Break & Continue

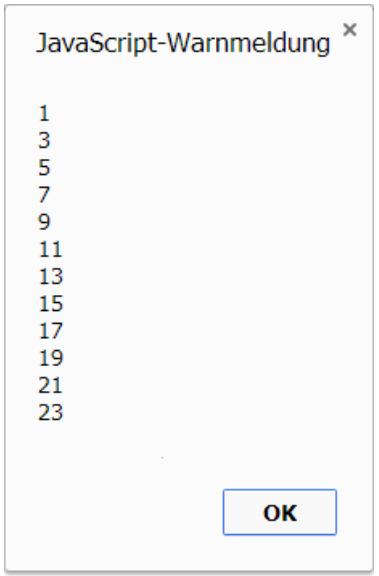
Bricht die innerste übergeordnete Schleife ab.

```
var a = 25, ausgabe = "";
while (a > 0) {
    a--;
    if (a == 17) break;
    ausgabe += a+"\n";
}
alert(ausgabe);
```



Bricht den aktuellen Schritt der innersten übergeordneten Schleife ab und macht mit dem nächsten weiter.

```
var ausgabe = "";
for (var a = 0; a < 25; a++) {
    if (a % 2 == 0) {
        continue;
    }
    ausgabe += a+"\n";
}
alert(ausgabe);
```



break und ***continue*** sind unabhängig von der Schleifenart!

Anwendung: Summe

```
<html>
<head>
    <title>Infocamp 2014</title>
    <script language="JavaScript">
        // Variablen deklarieren und initialisieren
        var x = 0, erg1 = 0, erg2 = 0;

        // Einlesen von X
        x = parseFloat(prompt("Summe von 1 bis..."));

        // Berechnen der Summe
        var i=1;
        while (i <= x) {
            erg1 += i;
            i++;
        }

        while (x > 0) {
            erg2 += x;
            x--;
        }

        // Ausgabe
        alert("Summe 1: " + erg1 + "\nSumme 2: " + erg2);
    </script>
</head>
<body>
    <!-- HTML Anweisungen -->
</body>
</html>
```

Zählvariable **i**
x unverändert!

Zählvariable **x**
x wird verändert!



3. Übung – Schleifen

Mögliche Projekte:

- ✓ Berechne die Summe der (geraden) Zahlen bis x
- Fülle einen Array (Länge y) mit 1en
- Palindrome: Eingabe, String "spiegeln", Ausgabe

```
while (<boolescher Ausdruck>) {
        <Anweisungen>;
}
```

```
do
{
        <Anweisungen>;
}while (<boolescher Ausdruck>);
```

```
for (<Initialisierung>;<Bedingung>;<Fortsetzung>) {
        <Anweisungen>;
}
```

Funktionen

Funktionen dienen dazu, Code-Abschnitte zusammenzufassen, um sie an beliebigen Stellen einfach per Aufruf über deren Namen **wiederverwenden** zu können.

Einer Funktion können Parameter übergeben werden, die sie zum Arbeiten braucht – ähnlich einer mathematischen Funktion.

- Berechnung und Rückgabe von Werten
- Ausführen von Anweisungen ohne Rückgabewert
- Erstellen von Objekten (später)

```
<html>
<head>
    <title>Infocamp 2014</title>
    <script language="JavaScript">
        var zahl1, zahl2, operator, ergebnis = 0;

        // Einlesen
        zahl1 = parseFloat(prompt("Zahl 1 eingeben:"));
        zahl2 = parseFloat(prompt("Zahl 2 eingeben:"));
        operator = prompt("Operator eingeben (+ - * / bis min max):");

        // Berechnen
        if(operator == "+") ergebnis = zahl1 + zahl2;
        else if(operator == "-") ergebnis = zahl1 - zahl2;
        else if(operator == "*") ergebnis = zahl1 * zahl2;
        else if(operator == "/") ergebnis = zahl1 / zahl2;
        else if(operator == "bis") {
            ergebnis = 0;
            for(var i=zahl1;i<=zahl2;i++){
                ergebnis += i;
            }
        }
        else if(operator == "max") {
            if (zahl1 > zahl2) ergebnis = zahl1;
            else ergebnis = zahl2;
        }
        else if(operator == "min") {
            if (zahl2 > zahl1) ergebnis = zahl1;
            else ergebnis = zahl2;
        }
        // Ausgabe
        alert("Das Ergebnis ist " + ergebnis);
    </script>
</head>
<body>
    <!-- HTML Anweisungen -->
</body>
</html>
```

Funktionen

```
function <name>(<parameter>) {
    <Anweisungen>;
    return <Wert oder Ausdruck>;
}
```

return optional

```
function ErgebnisAusgabe(e){
    alert("Das Ergebnis ist "+e);
}
function Addition(a, b){
    return a + b;
}
function Eingabe(i) {
    return parseFloat(prompt("Bitte Zahl "+ i +" eingeben: "));
}
```

Funktionsaufruf

```
function ErgebnisAusgabe(e){
    alert("Das Ergebnis ist "+e);
}
function Addition(a, b){
    return a + b;
}
function Eingabe(i) {
    return parseFloat(prompt("Bitte Zahl "+ i +" eingeben: "));
}

var zahl1, zahl2, ergebnis = 0;

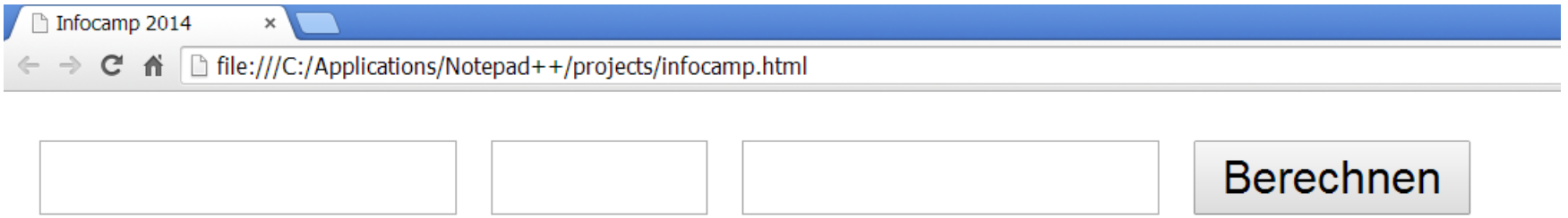
// Einlesen
zahl1 = Eingabe(1);
zahl2 = Eingabe(2);

// Berechnen
ergebnis = Addition(zahl1, zahl2);

// Ausgabe
ErgebnisAusgabe(ergebnis);
```

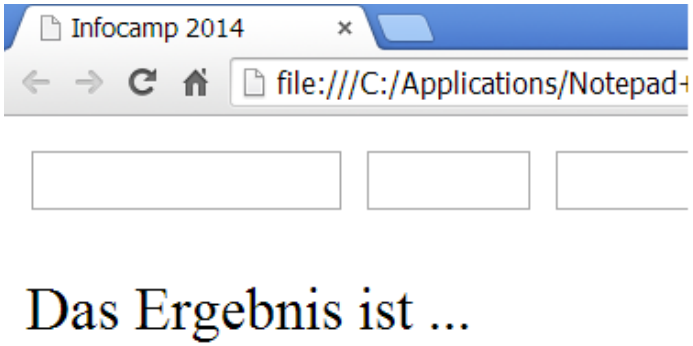
Einschub: HTML & JavaScript

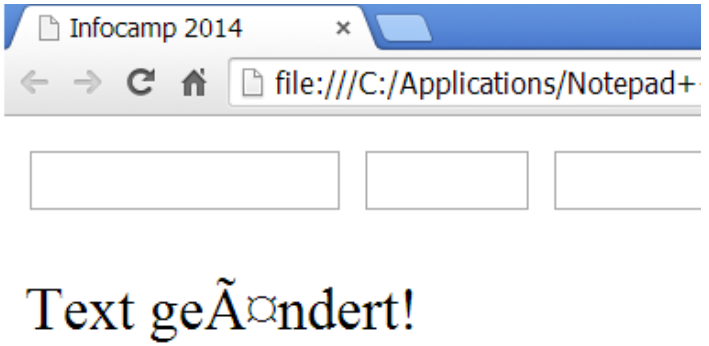
```
<body>
    <input id="zahl1" type="text" size="15" maxlength="15"/>
    <input id="operator" type="text" size="5" maxlength="5"/>
    <input id="zahl2" type="text" size="15" maxlength="15"/>
    <input type="button" onclick="Execute()" value="Berechnen"/>
    <p id="ausgabe">Das Ergebnis ist ...</p>
</body>
```



Einschub: HTML & JavaScript

```
<html>
<head>
    <title>Infocamp 2014</title>
    <script language="JavaScript">
        function textChange(){
            document.getElementById("ausgabe").innerHTML = "Text geändert!";
        }
    </script>
</head>
<body>
    <input id="zahl1" type="text" size="15" maxlength="15"/>
    <input id="operator" type="text" size="5" maxlength="5"/>
    <input id="zahl2" type="text" size="15" maxlength="15"/>
    <input type="button" onclick="Execute()" value="Berechnen"/>
    <p id="ausgabe" onclick="textChange()">Das Ergebnis ist ...</p>
</body>
</html>
```





```
<html>
<head>
    <title>Infocamp 2014</title>
    <script language="JavaScript">
        function ErgebnisAusgabe(e){
            document.getElementById("ausgabe").innerHTML = "Das Ergebnis ist "+e;
        }
        function Addition(a, b){
            return a + b;
        }
        function Subtraktion(a, b){
            return a - b;
        }
        function Multiplikation(a, b){
            return a * b;
        }
        function Division(a, b){
            return a / b;
        }
        function Summe(a, b){
            if (b < a) return 0;
            var erg = 0;
            for(var i=a;i<=b;i++){
                erg += i;
            }
            return erg;
        }
        function Min(a, b){
            if (a > b) return b;
            else return a;
        }
        function Max(a, b){
            if (a > b) return a;
            else return b;
        }
        function Eingabe(id) {
            return parseFloat(document.getElementById(id).value);
        }
        function Execute(){
            var zahl1, zahl2, operator, ergebnis = 0;

            // Einlesen
            zahl1 = Eingabe("zahl1");
            zahl2 = Eingabe("zahl2");
            operator = document.getElementById("operator").value;

            // Berechnen
            if(operator == "+") ergebnis = Addition(zahl1, zahl2);
            else if(operator == "-") ergebnis = Subtraktion(zahl1, zahl2);
            else if(operator == "*") ergebnis = Multiplikation(zahl1, zahl2);
            else if(operator == "/") ergebnis = Division(zahl1, zahl2);
            else if(operator == "bis") ergebnis = Summe(zahl1, zahl2);
            else if(operator == "max") ergebnis = Max(zahl1, zahl2);
            else if(operator == "min") ergebnis = Min(zahl1, zahl2);

            // Ausgabe
            ErgebnisAusgabe(ergebnis);
        }
    </script>
</head>
<body>
    <input id="zahl1" type="text" size="15" maxlength="15"/>
    <input id="operator" type="text" size="5" maxlength="5"/>
    <input id="zahl2" type="text" size="15" maxlength="15"/>
    <input type="button" onclick="Execute()" value="Berechnen"/>
    <p id="ausgabe">Das Ergebnis ist ...</p>
</body>
</html>
```

4. Übung – Methoden

Projekt:

- Auslagern der Funktionen der 3. Übung in Methoden

```
function <name>(<parameter>) {
    <Anweisungen>;
    return <Wert oder Ausdruck>;
}
```

Aufruf:

```
<methodenname>(<wert1>, <wert2>);
var <variablenname> = <methodenname>(<wert>);
```

Objekte

Objekte sind Sammlungen von Variablen und Funktionen,
die bestimmte Konstrukte beschreiben.

Beispiel aus der Mathematik → Bruch

Ein Bruch besteht aus:

- **Variablen:**
  - Zähler
  - Nenner
- **Funktionen:**
  - erweiternMit
  - getDecimal
  - toString → Darstellen als Zeichenkette (z.B. "2/3")

Objekt erstellen

Drei Möglichkeiten Objekte zu erstellen:

1. Erstellen eines Objekts, dann Anfügen von Eigenschaften

```
var einhalb = {};
einhalb.zaehler = 1;
einhalb.nenner = 2;
```

Objekt erstellen

Drei Möglichkeiten Objekte zu erstellen:

2. Direktes Auflisten der Eigenschaften mit jew. Werten

```
var einhalb = { zaehler : 1,
                nenner  : 2 };
```

Objekt erstellen

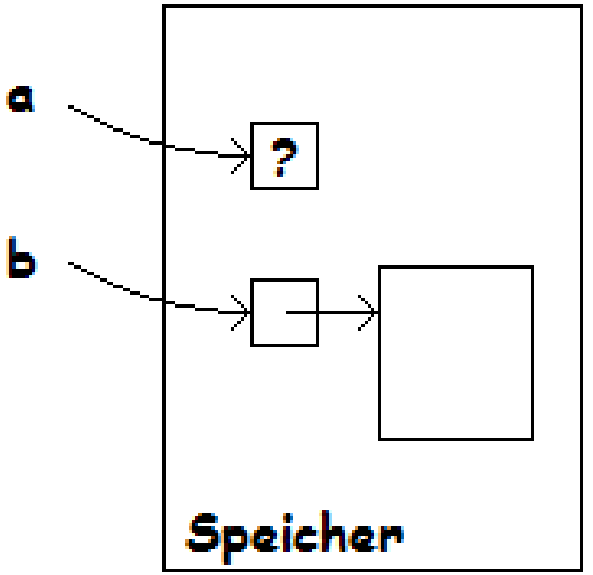
Drei Möglichkeiten Objekte zu erstellen:

3. Festlegen einer Konstruktor-Funktion

```
function Bruch(z, n){
    this.zaehler = z;
    this.nenner = n;
}
var einhalb = new Bruch(1,2);
```

Objekt erstellen

```
function Bruch(z, n){
    this.zaehler = z;
    this.nenner = n;
}
var a;
var b = new Bruch(1,2);
```



Objekte

Objekte sind variabel → jederzeit erweiterbar!
(siehe Möglichkeit 1 zum Erstellen)

```
function Bruch(z, n){
    this.zaehler = z;
    this.nenner = n;
}
var einhalb = new Bruch(1,2);
var dreiviertel = new Bruch(3,4);
einhalb.dezimal = einhalb.zaehler / einhalb.nenner;

alert(einhalb.dezimal);          // Gibt "0,5" aus
alert(dreiviertel.dezimal);      // Gibt "undefined" aus
```

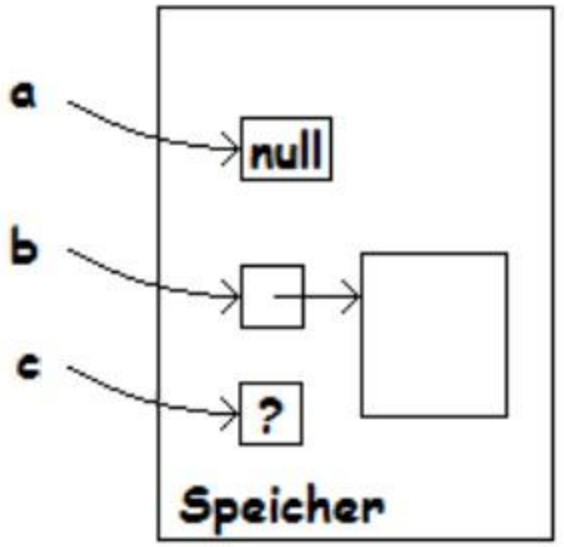
"null" - Referenz

```
var a = null;
var b = new Bruch(1,2);
var c;

if (a == null) {
    // mach was
}

if (b != null) {
    // mach was
}

if (c == undefined) {
    // mach was
}
```



Objekt-Funktionen

this.<name> = ***function***(<parameter>) {
    <Anweisungen>;
    return <Wert oder Ausdruck>;
}

```
function Bruch(z, n){
    this.zaehler = z;
    this.nenner = n;

    this.getDecimal = function(){
        return this.zaehler / this.nenner;
    }
    this.erweiternMit = function(x){
        this.zaehler *= x;
        this.nenner *= x;
    }
    this.toString = function(){
        return this.zaehler + "/" + this.nenner;
    }
}
```

Objekte an Funktionen geben

```
this.addBruch = function(b){
    this.zaehler = this.zaehler * b.nenner + b.zaehler * this.nenner;
    this.nenner = this.nenner * b.nenner;
}
```



```
this.addBruch = function(b){
    if(b.nenner == undefined || b.zaehler == undefined){
        return;
    }
    this.zaehler = this.zaehler * b.nenner + b.zaehler * this.nenner;
    this.nenner = this.nenner * b.nenner;
}
```

5. Übung – Klassen & Objekte

Entwickeln eines Objektes "Vector3“, das folgendes leisten soll:

- Eigenschaften:
  - 3 Koordinaten (x,y,z)
- Funktionen:
  - Länge des Vektors berechnen
  - Vektor normalisieren
  - Skalar- & Kreuzprodukte zweier Vektoren bilden
  - Vektor mit einem Skalar multiplizieren / dividieren
  - Addition / Subtraktion zweier Vektoren

```
// Definieren
function <objektname>(<params>){
    // Objektvariablen
    this.<name> = <wert>;

    // Objektfunktion
    this.<name> = function(<params>) {
        <Anweisungen>;
        return <Wert oder Ausdruck>;
    }
}
```

```
// Erstellen
var <name> = new <objektname>(<params>);

// Verwenden von Variablen
<name>.<variablenname> = 3;
alert(<name>.<variablenname>);

// Verwenden von Funktionen
<name>.<funktionsname>(<params>);
var x = <name>.<funktionsname>(<params>);
```